TECNOLOGIAS WEB APLICADAS AO DESENVOLVIMENTO DE UM SISTEMA DE GESTÃO PARA SERRALHERIA

Heitor Venâncio Marins da Silva, Jaime William Dias

Universidade Paranaense (Unipar) – Unidade de Paranavaí
Paranavaí – PR – Brasil

Resumo. A escolha das tecnologias influencia diretamente a segurança, a manutenção e a escalabilidade de um sistema web. Este artigo tem como objetivo apresentar e detalhar as tecnologias empregadas no desenvolvimento de um sistema de gestão para uma serralheria, descrevendo o papel de cada uma e a forma como se integram. A metodologia combinou pesquisa nas documentações oficiais com a aplicação prática em desenvolvimento incremental, utilizando Java, Spring Boot, Spring Security, PostgreSQL, React e WPPConnect. Conclui-se que o conjunto adotado, de código aberto e amplamente documentado, permitiu construir uma aplicação segura, organizada em camadas e de fácil manutenção.

1. Introdução

O desenvolvimento de sistemas web envolve a combinação de diversas tecnologias: uma linguagem e um framework para o servidor, um banco de dados, uma biblioteca para a interface e ferramentas de apoio. A escolha desse conjunto define a segurança da aplicação, o esforço de manutenção e a facilidade de evolução do software [Sommerville, 2011].

Nesse contexto, este artigo tem como objetivo apresentar e detalhar as tecnologias utilizadas no desenvolvimento de um sistema web de gestão para a HSA Serralheria, empresa do ramo metalúrgico de Paranavaí-PR. O foco está no papel de cada tecnologia e na forma como elas se integram; o sistema construído é apresentado como resultado da aplicação desse conjunto.

O artigo está organizado da seguinte forma: a Seção 2 descreve a metodologia; a Seção 3 detalha a arquitetura e as tecnologias do back-end, do banco de dados, do front-end e da integração; a Seção 4 apresenta os resultados obtidos; e a Seção 5, as considerações finais.

2. Metodologia

A pesquisa foi de natureza aplicada. Inicialmente, realizou-se um estudo das documentações oficiais das tecnologias candidatas, adotando como critérios de escolha o licenciamento de código aberto, a maturidade, o tamanho da comunidade e a compatibilidade entre as ferramentas.

Em seguida, as tecnologias escolhidas foram aplicadas no desenvolvimento do sistema, seguindo o modelo incremental [Pressman e Maxim, 2016]. A modelagem foi feita com a UML [Booch et al., 2006] na ferramenta PlantUML; cada incremento teve seus endpoints testados no Postman antes da integração com a interface, e o código foi

versionado com Git.

3. Tecnologias Utilizadas

Esta seção apresenta a arquitetura adotada e detalha as tecnologias do back-end, do banco de dados, do front-end e da integração, resumidas ao final na Tabela 1.

3.1. Arquitetura Cliente-Servidor e API REST

O sistema adota a arquitetura cliente-servidor, com o front-end e o back-end desenvolvidos como aplicações independentes. A comunicação segue o estilo arquitetural REST [Fielding, 2000]: o cliente acessa recursos identificados por URLs usando os métodos HTTP (GET, POST, PUT e DELETE) e troca dados no formato JSON. Como o REST é sem estado, cada requisição carrega as informações necessárias para ser atendida, inclusive o token de autenticação.

No servidor, o código é organizado em camadas: os controllers recebem as requisições HTTP, os services concentram as regras de negócio e os repositories acessam o banco de dados. A Figura 1 mostra o caminho de uma requisição entre as tecnologias.

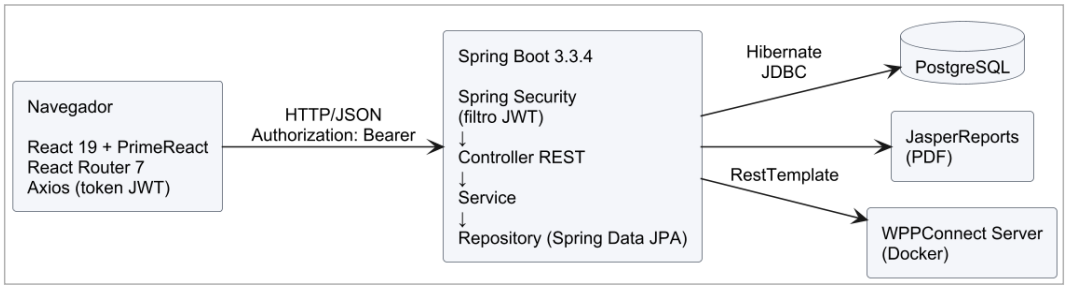


Figura 1: Integração entre as tecnologias do sistema

3.2. Back-end

O back-end é a aplicação executada no servidor, responsável pelas regras de negócio, pela segurança e pelo acesso ao banco de dados, expondo a API REST consumida pela interface.

3.2.1. Java

Java é uma linguagem orientada a objetos cujo código é compilado para bytecode e executado na Java Virtual Machine (JVM), o que garante portabilidade entre sistemas operacionais [Oracle, 2026]. Foi utilizada a versão 21, de suporte estendido (LTS), que recebe atualizações de segurança por longo prazo. O projeto usa o Maven para gerenciar dependências e compilar a aplicação, e o Lombok para gerar automaticamente métodos de acesso e construtores a partir de anotações.

3.2.2. Spring Boot

O Spring Boot é construído sobre o Spring Framework, cujo núcleo é a inversão de controle: o framework cria e injeta os objetos (beans) de que cada classe precisa, reduzindo o acoplamento. O Spring Boot acrescenta a autoconfiguração, os starters, que agrupam dependências por finalidade, e um servidor Tomcat embutido, que permite executar a aplicação como um único arquivo sem instalar um servidor à parte

[Spring, 2026a].

Na versão 3.3.4 adotada, foram usados os starters web, data-jpa, security, mail e freemarker. Os endpoints são criados com a anotação @RestController, e o agendamento de tarefas com @Scheduled executa rotinas automáticas em horários definidos. O envio de e-mails usa o JavaMail com modelos de mensagem FreeMarker.

3.2.3. Spring Data JPA e Hibernate

A JPA (Java Persistence API) é a especificação Java para mapeamento objeto-relacional, e o Hibernate é a implementação utilizada [Hibernate, 2026]. As classes anotadas com @Entity são convertidas em tabelas, e o Hibernate traduz as operações sobre objetos em comandos SQL para o dialeto do PostgreSQL.

O Spring Data JPA simplifica essa camada: basta declarar uma interface que estende JpaRepository para obter as operações de inserção, consulta, alteração e exclusão, e consultas adicionais podem ser derivadas do nome do método [Spring, 2026c]. Com a opção ddl-auto=update, o esquema do banco é atualizado a partir das entidades.

3.2.4. Spring Security e JWT

O Spring Security trata a autenticação e a autorização por meio de uma cadeia de filtros que intercepta as requisições [Spring, 2026b]. No projeto, ele foi configurado sem sessão no servidor (stateless), e as senhas são armazenadas com o algoritmo de hash BCrypt, que inclui um valor aleatório (salt) em cada senha.

A identidade do usuário é transportada por um JSON Web Token (JWT), padrão definido na RFC 7519 [Jones et al., 2015]. O token tem três partes codificadas em Base64URL: cabeçalho, conteúdo (claims, como usuário e validade) e assinatura. A biblioteca jjwt 0.12.6 gera o token no login, assinado com HMAC-SHA; a cada requisição, um filtro próprio (OncePerRequestFilter) valida a assinatura e a validade antes de liberar o acesso conforme o perfil.

3.2.5. Bibliotecas de Relatórios e Planilhas

O JasperReports 7.0.3 gera relatórios a partir de modelos no formato JRXML, compilados em arquivos .jasper [TIBCO Jaspersoft, 2026]. O modelo é preenchido com os dados do banco e exportado para PDF diretamente em memória, sem gravar arquivos no servidor. Já o Apache POI 5.3.0 é a API Java para documentos do Microsoft Office [Apache Software Foundation, 2026]; com a classe XSSFWorkbook, o sistema lê e gera planilhas .xlsx para importação de dados.

3.3. Banco de Dados: PostgreSQL

O PostgreSQL é um sistema gerenciador de banco de dados objeto-relacional de código aberto, com suporte a transações ACID, integridade referencial por chaves estrangeiras e ampla aderência ao padrão SQL [PostgreSQL Global Development Group, 2026]. A aplicação se conecta a ele pelo driver JDBC, e o acesso aos dados passa pelo Hibernate, que gera a maior parte dos comandos SQL.

3.4. Front-end

O front-end é a aplicação executada no navegador do usuário, responsável pela interface e pelo consumo da API REST do back-end.

3.4.1. JavaScript e React

O front-end foi desenvolvido em JavaScript com o React 19, biblioteca para construção de interfaces baseada em componentes reutilizáveis [Meta Platforms, 2026]. O React é declarativo: o desenvolvedor descreve como a tela deve ficar para cada estado, e a biblioteca atualiza apenas as partes que mudaram. O estado dos componentes é controlado com hooks, como useState e useEffect.

O projeto foi criado com o Create React App e usa o NPM, gerenciador de pacotes do Node.js, para instalar as dependências. A aplicação funciona como uma SPA (single-page application): o React Router 7 troca as páginas sem recarregar o navegador e bloqueia rotas conforme o perfil do usuário.

3.4.2. PrimeReact, Material UI e Axios

O PrimeReact 10 fornece componentes prontos, como tabelas com paginação e filtros, formulários, caixas de diálogo e notificações, além do conjunto de ícones PrimeIcons [PrimeTek, 2026]; o Material UI 6 complementa alguns elementos visuais. Essas bibliotecas reduzem o tempo de construção das telas e mantêm a interface padronizada.

A comunicação com a API usa o Axios 1.8, cliente HTTP baseado em Promises [Axios, 2026]. Foi criada uma instância com o endereço base da API e um interceptor que adiciona automaticamente o cabeçalho Authorization: Bearer com o token JWT em todas as requisições.

3.5. Integração com WhatsApp: WPPConnect e Docker

O WPPConnect Server é um projeto de código aberto, desenvolvido em Node.js, que expõe uma API REST para o WhatsApp Web [WPPConnect Team, 2026]. Ele é executado em um contêiner Docker, tecnologia que empacota a aplicação com todas as suas dependências e a isola do sistema operacional, garantindo que funcione da mesma forma em qualquer máquina [Docker Inc., 2026]. O back-end se comunica com o WPPConnect por HTTP, usando o RestTemplate do Spring, para enviar mensagens e arquivos PDF.

3.6. Ferramentas de Desenvolvimento

O código foi escrito no Visual Studio Code, editor com extensões para Java e JavaScript. O Postman foi usado para testar os endpoints da API de forma isolada, enviando requisições com o token JWT. O Git controlou as versões do código, e o PlantUML gerou os diagramas UML a partir de descrições em texto. A Tabela 1 resume as tecnologias adotadas, suas versões e a função de cada uma no projeto.

Tabela 1. Tecnologias utilizadas no projeto

Tecnologia	Versão	Função no projeto
Java	21 (LTS)	Linguagem do back-end
Spring Boot	3.3.4	Base da API REST, injeção de dependências e servidor embutido
Spring Data JPA / Hibernate	3.3.4 / 6.5	Mapeamento objeto-relacional e acesso ao banco
Spring Security / jjwt	6.3 / 0.12.6	Autenticação com token JWT e autorização por perfil
PostgreSQL	—	Sistema gerenciador de banco de dados relacional
React	19	Biblioteca de interface do front-end
PrimeReact / Material UI	10 / 6	Componentes visuais prontos
Axios / React Router	1.8 / 7	Requisições HTTP e navegação entre páginas
JasperReports	7.0.3	Geração de relatórios em PDF
Apache POI	5.3.0	Leitura e escrita de planilhas .xlsx
WPPConnect Server / Docker	—	API de integração com o WhatsApp em contêiner
Maven / NPM	—	Gerenciamento de dependências e build
VS Code / Postman / Git	—	Edição de código, testes de API e versionamento

4. Resultados Obtidos

O resultado da aplicação dessas tecnologias foi o sistema HSA Serralheria, uma aplicação web em funcionamento cuja interface é mostrada na Figura 2. O back-end reúne 26 controllers REST, 36 services e 30 entidades JPA persistidas no PostgreSQL, e o front-end possui 20 páginas React.

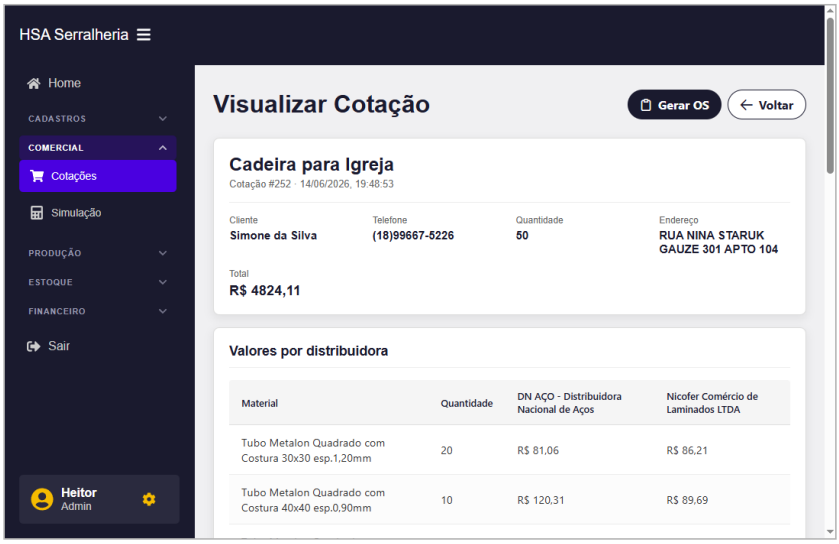


Figura 2: Interface do sistema desenvolvido

Na prática, a autoconfiguração do Spring Boot e os repositories do Spring Data JPA reduziram o código de infraestrutura, permitindo concentrar o esforço nas regras de negócio. O uso de JWT sem sessão simplificou a separação entre front-end e

back-end, e os componentes do PrimeReact padronizaram as telas. A execução do WPPConnect em Docker evitou a instalação manual do Node.js e de suas dependências na máquina do servidor.

5. Considerações Finais

O conjunto formado por Java, Spring Boot, PostgreSQL e React mostrou-se adequado ao desenvolvimento de um sistema web de gestão: todas as tecnologias são de código aberto, possuem documentação ampla e se integram de forma consistente por meio da API REST. A separação em camadas e o uso de padrões como JWT e JPA facilitam a manutenção e a evolução do software.

Como trabalhos futuros, propõem-se a execução de toda a aplicação em contêineres Docker, a criação de testes automatizados e a publicação do sistema em um serviço de nuvem.

Referências

Apache Software Foundation (2026) Apache POI: the Java API for Microsoft Documents. Disponível em: <https://poi.apache.org/>. Acesso em: 27/09/2026.

Axios (2026) Axios Documentation. Disponível em: <https://axios-http.com/>. Acesso em: 27/09/2026.

Booch, G. et al. (2006) UML: guia do usuário. 2. ed. Rio de Janeiro: Elsevier.

Docker Inc. (2026) Docker Documentation. Disponível em: <https://docs.docker.com/>. Acesso em: 27/09/2026.

Fielding, R. T. (2000) Architectural Styles and the Design of Network-based Software Architectures. Tese (Doutorado em Ciência da Informação e Computação). University of California, Irvine.

Hibernate (2026) Hibernate ORM Documentation. Disponível em: <https://hibernate.org/orm/documentation/>. Acesso em: 27/09/2026.

Jones, M. et al. (2015) RFC 7519: JSON Web Token (JWT). Internet Engineering Task Force (IETF). Disponível em: <https://www.rfc-editor.org/rfc/rfc7519>. Acesso em: 27/09/2026.

Meta Platforms (2026) React: the library for web and native user interfaces. Disponível em: <https://react.dev/>. Acesso em: 27/09/2026.

Oracle (2026) Java Platform, Standard Edition. Disponível em: <https://www.oracle.com/java/technologies/javase/>. Acesso em: 27/09/2026.

PostgreSQL Global Development Group (2026) PostgreSQL Documentation. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 27/09/2026.

Pressman, R. S. e Maxim, B. R. (2016) Engenharia de software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH.

PrimeTek (2026) PrimeReact. Disponível em: <https://primereact.org/>. Acesso em: 27/09/2026.

Sommerville, I. (2011) Engenharia de software. 9. ed. São Paulo: Pearson Prentice Hall.

Spring (2026a) Spring Boot Reference Documentation. Disponível em: <https://docs.spring.io/spring-boot/>. Acesso em: 27/09/2026.

Spring (2026b) Spring Security Reference. Disponível em: <https://docs.spring.io/spring-security/reference/>. Acesso em: 27/09/2026.

Spring (2026c) Spring Data JPA Reference Documentation. Disponível em: <https://docs.spring.io/spring-data/jpa/reference/>. Acesso em: 27/09/2026.

TIBCO Jaspersoft (2026) JasperReports Library. Disponível em: <https://community.jaspersoft.com/project/jasperreports-library>. Acesso em: 27/09/2026.

WPPConnect Team (2026) WPPConnect Server. Disponível em: <https://github.com/wppconnect-team/wppconnect-server>. Acesso em: 27/09/2026.